

CAPSTONE PROJECT · AGENTIC AI

AI IT Support Assistant

An agentic help desk that answers, looks up, and raises tickets —
but never writes without permission.

LangGraph orchestration · 5 tools · SQLite · BM25 retrieval · Streamlit

Local-first: runs with Ollama. Claude support builtin.

THE PROBLEM

Service desks drown in a handful of requests

Repetitive, but not scripted - which is exactly why scripts and chatbots fail at them.

Same intent, different words

“VPN keeps dropping”, “can't connect from home”,
“remote access broken” — one problem, many phrasings.
Keyword bots miss it; a human re-reads the same article.

Details always missing

The employee says “raise a ticket” and nothing else.
No ID, no category, no description.
An agent that acts anyway creates junk.

Some actions must not be blind

A duplicate ticket is worse than no ticket: it splits history and wastes a second engineer.
Writes need consent and a duplicate check.

The gap

Language understanding is the easy half. The hard half is policy - knowing when to refuse to act yet.

THE PRODUCT

Three jobs, one conversation

Everything runs against the organisation's own data - no public internet, no invented facts.

01

Answer
Searches 15 IT how-to articles with BM25 and answers in plain language.
No match → it says so.

02

Look up
Finds existing tickets by employee, ID, status or keyword, and reports live service health.

03

Raise
Creates a ticket - only with every required field, the employee's consent, and no open duplicate.

THE MULTI-TURN FLOW THAT PROVES IT

User

I have a VPN issue.

Assistant

Known degradation on the EMEA VPN gateway. What is your employee ID?

system_status

User

EMP1024.

Assistant

Found you, Priya Sharma (Finance, Bengaluru). Check your open tickets?

get_employee

User

Yes.

Assistant

You have INC-10001 (In Progress) for VPN drops.

ticket_lookup

The model decides. The workflow enforces.

An LLM picks the intent and the wording. A graph around it holds the rules the model cannot be trusted to hold.

Trusted to the model

- Reading the request in any phrasing
- Choosing which tool fits
- Writing the answer a human reads

Taken away from the model

- Whether a write is allowed to run
- Extracting the employee ID (regex)
- Detecting a duplicate ticket (SQL)
- Confirmation detection, input limits

Reads run freely

Knowledge, tickets, employees, status execute the moment the model asks.

The one write is gated

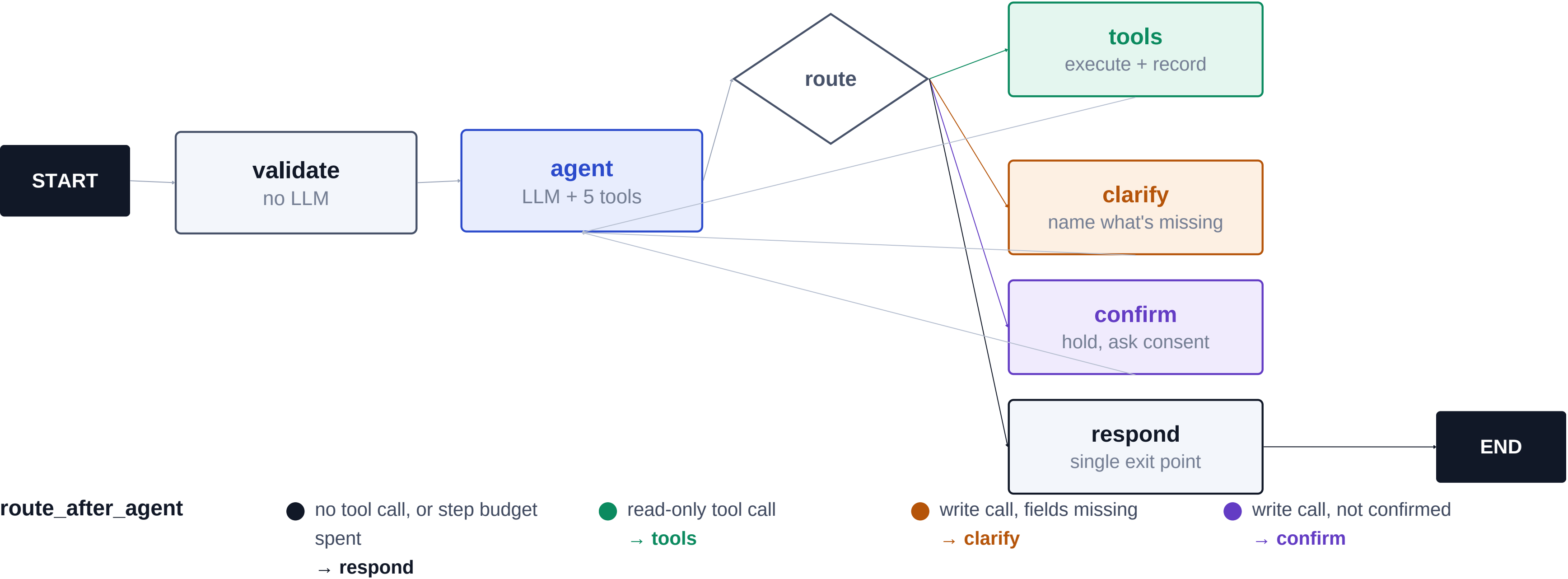
create_ticket is intercepted every time - clarify if incomplete, confirm if unapproved.

Nothing is unexplained

Every tool call and its raw result is shown, so facts are separable from phrasing.

The LangGraph workflow

Six nodes. One conditional edge does all the policy work.



What each node is responsible for

Only one of six nodes calls the model. Everything a regex or a SQL query does better stays in Python.

NODE	LLM?	RESPONSIBILITY
validate	No	Rejects empty or >2000-char input, extracts EMP#####, reads a confirmation or a refusal.
agent	Yes	Decides whether a tool is needed and with which arguments. Wraps model outages.
tools	No	Executes every tool call, records a tool_event, promotes a lookup into remembered state.
clarify	No	Answers an incomplete create_ticket call by naming exactly the missing fields.
confirm	No	Human-in-the-loop gate. Stores the draft and asks; writes nothing.
respond	No	Single exit point. Guarantees a non-empty answer.

Why it matters: a reviewer can point at the node that enforces a rule. Prompt instructions cannot be pointed at.

State: what it remembers, and what it forgets

Typed channels with explicit reducers, checkpointed per conversation thread.

CHANNEL	REDUCER	PURPOSE
messages	add_messages	Running transcript.
employee_id / profile	last value	Identity, remembered across turns.
pending_ticket	last value	Draft held back, awaiting consent.
confirmed	last value	True only for the turn the user agreed.
missing_fields	last value	What a ticket still needs.
tool_events	operator.add	Append-only audit trail rendered by the UI.
steps	last value	Tool-loop budget for the turn (MAX_AGENT_STEPS = 6).

Memory that pays off

“yes, check my tickets” resolves three turns after the ID was given - no re-asking. SqliteSaver persists the thread, so a page refresh does not lose the conversation.

Memory that is dropped on purpose

A new EMP##### clears the remembered profile and any pending draft, rather than re-attributing it. A “yes” that also names a new employee does not stand as consent for the previous one's ticket.

Five tools, four read, one writes

Native function calling via langchain-core. Every tool returns an envelope, never an exception.

TOOL	KIND	PURPOSE	BACKING TABLE
knowledge_search	Read	BM25 search over IT how-to articles	kb_articles (15)
ticket_lookup	Read	Tickets by employee, ID, status or keyword	tickets (18)
get_employee	Read	Resolve an employee from ID, email or name	employees (12)
system_status	Read	Current health of internal services	system_status (6)
create_ticket	WRITE	Raise a ticket — validated, confirmed, de-duplicated	tickets

The envelope contract

{ ok: bool, error: str | None, ...payload }

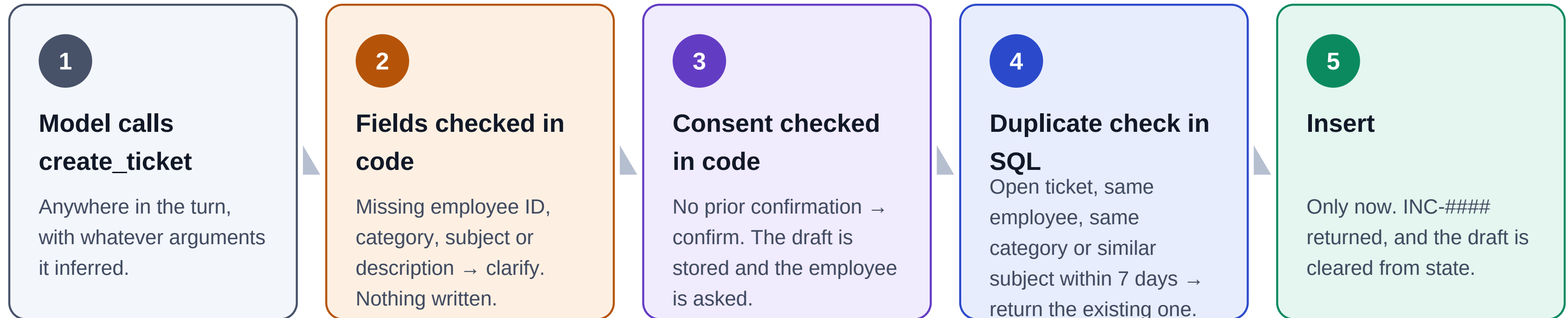
Every tool is wrapped: an unexpected exception becomes a failed result, not a stack trace.

No fabrication

The prompt forbids inventing tickets, employees or articles. A search below the relevance floor returns an explicit “no article matched” rather than the closest bad answer.

The write gate, step by step

`create_ticket` never executes on the turn the model first asks for it.



The gate is code, not a prompt

The prompt asks the model to confirm. `route_after_agent` guarantees it - when the model jumps the gun, the graph answers its tool call with an instruction, so the transcript stays valid and no row is written.

Six controls, each testable on its own

Every one of these has a test that fails if the control is removed.

1 Write gate

create_ticket is the only write, and it is intercepted on every turn.

2 Field validation

Required fields, employee existence, allowed category and priority vocabularies.

3 Duplicate suppression

Open ticket, same employee, same category or similar subject inside the 7-day window.

4 No fabrication

Structured tool results; a no-match search says so explicitly.

5 Graceful failure

Tool exceptions become failed results; a model outage becomes a plain sentence.

6 Loop budget

MAX_AGENT_STEPS caps tool calls per turn, so no runaway loop.

Failure direction is always chosen: an ambiguous “that would be grand” reads as a non-confirmation, and the assistant asks again.

Four turns that show the policy working

Real transcript fragments - tool call, then the answer the employee sees.

KNOWLEDGE SEARCH

“How do I reset my VPN password?”

knowledge_search → KB-001 (score 7.78)

Open portal.sourcify.example/vpn, choose Reset VPN Credentials, clear MFA, set a 14-char password.

MISSING INFORMATION

“Raise a VPN ticket for me.”

route → clarify (nothing written)

I need a few details first: your employee ID, a short subject, and a description of the problem.

CONFIRMATION

“Monitor flickers, raise a ticket. I am EMP1128.”

route → confirm (draft held)

Confirm: EMP1128 · Hardware · “Monitor flickering constantly”. Shall I go ahead?

DUPLICATE SUPPRESSION

“Outlook is broken again, raise a ticket. EMP1042.”

create_ticket → created: false

You already have an open Email ticket, INC-10004, with the Messaging Team. I have not raised a second one.

Stack, and why each piece was chosen

Every choice optimised for a project that must run and be inspected without a network.

LAYER	CHOICE	WHY
Orchestration	LangGraph - StateGraph, conditional edges, SqliteSaver	Policy becomes a visible node, not a prompt line
LLM	Ollama qwen3.5:4b-mlx (default) / Anthropic Claude	Runs with no API key; provider is a one-line switch
Tool calling	langchain-core tools, native function calling	Typed args, no prompt-parsed JSON
Retrieval	BM25 in pure Python	15 articles need no vector store; deterministic and inspectable
Storage	SQLite, seeded from JSON	Zero setup, real SQL for the duplicate check
UI	Streamlit chat + read-only browse module	Browsing is a human activity; it bypasses the model
Config / tests	pydantic-settings + .env; pytest	65 tests, no network, no API key

LangGraph over a plain tool loop

The interesting logic is not “call a tool”, it is “refuse to call this tool yet”.

BM25 over embeddings

No dependency, no API cost, no drift — and a relevance floor turns weak matches into “no match”.

A workspace built for an evaluator

Three columns: the session, the conversation, and the raw data the assistant works from.

Left — session

- Active model and thread ID
- Clear conversation
- Four sample prompts
- The five reachable tools

Middle — conversation

- Identity banner: who the assistant thinks it is helping
- EMP#### and INC-##### highlighted inline
- Touched tickets as status/priority chip cards
- Expandable trace: every call + raw JSON

Right — the local data

- Knowledge base: 15 articles, live BM25 filter, scores
- Tickets: status counters, filter by status/category
- Directory & status: 12 employees, 6 services

Provenance, not trust
A claim in an answer can be checked against the source table without leaving the page - retrieved facts are visibly separate from the model's own recommendation.

65 tests, and they run anywhere

A temporary database and a scripted stand-in for the model: no Ollama, no API key, no network.

65

tests passing

2.0s

full suite runtime

0

network calls required

5/5

routing branches covered

AREA	WHAT IS COVERED
Retrieval	BM25 ranking and the relevance floor
Tools	All five, plus validation and duplicate rules, plus induced failures
Routing	Every branch of route_after_agent and route_after_validate
Memory	Slot retention across turns; what is dropped on an employee switch
Gates	Both confirmation paths; empty and oversized input; a model outage
UI	streamlit.testing.v1.AppTest runs app.py in-process, so a rendering error fails the suite

Known limits, and what comes next

Each limit fails in the safe direction: surface the problem rather than act on a guess.

Limitations

- Small local model over-explains or re-checks; the gates hold regardless.
- Confirmation is keyword-based - an unusual “yes” reads as a non-confirmation.
- Duplicate detection is heuristic; two real VPN problems in a week get flagged.
- Single-user demo: SQLite, no auth, the employee ID is taken on trust.
- Tickets can be created and read, not updated or closed.

Roadmap

- Ticket update and close, behind the same confirm gate.
- Real identity from SSO, so the employee ID is proven, not typed.
- An LLM-judged duplicate check on top of the SQL heuristic.
- Escalation routing by category instead of a default Service Desk.
- Answer-quality evaluation set, run in CI alongside the unit tests.

THE ONE-LINE VERSION

**An LLM good enough to understand the request,
inside a workflow strict enough to be trusted with a write.**

DEMO IDS

EMP1024

two VPN tickets, one open

EMP1115

open laptop hardware ticket

EMP1067

no VPN entitlement

RUN IT

```
pip install -r requirements.txt  
python scripts/seed_db.py  
streamlit run app.py  
python -m pytest -q
```